

MonarchAttention: Sub-Quadratic Attention via Monarch Matrices

Setup

For queries, keys, and values $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$, one head of softmax attention computes

$$\mathbf{O} = \text{softmax}(\mathbf{Q}\mathbf{K}^\top)\mathbf{V}, \quad (1)$$

with softmax applied across rows. Forming the $N \times N$ attention matrix costs $\Theta(N^2d)$. The goal of MonarchAttention is to find a structured matrix $\mathbf{M} \approx \text{softmax}(\mathbf{Q}\mathbf{K}^\top)$ in $o(N^2d)$ time, without ever materializing the full attention matrix, and then compute $\mathbf{O} \approx \mathbf{M}\mathbf{V}$ cheaply.

Throughout, a summation without written indices runs over all indices in its summand.

Preliminaries

Variational form of softmax. For $\mathbf{z} \in \mathbb{R}^N$ and temperature $c > 0$,

$$\text{softmax}(\mathbf{z}/c) = \arg \max_{\mathbf{p} \in \Delta^N} \sum_n \mathbf{p}_n \mathbf{z}_n - c \sum_n \mathbf{p}_n \log \mathbf{p}_n, \quad (2)$$

where Δ^N is the probability simplex. Softmax is the unique maximizer of a linear term plus scaled entropy.

Monarch matrices. Let $N = mb$. A matrix $\mathbf{B} \in \mathbb{R}^{N \times N}$ is *block rank-one* if there are $\mathbf{L} \in \mathbb{R}^{b \times m \times m}$ and $\mathbf{R} \in \mathbb{R}^{m \times b \times b}$ with

$$\mathbf{B}_{(jl)(ki)} = \mathbf{L}_{jkl} \mathbf{R}_{kji}, \quad i, j \in [b], \quad k, l \in [m], \quad (3)$$

where $(jl) = (j-1)m + l$ and $(ki) = (k-1)b + i$. That is, $\mathbf{B}$ is a $b \times m$ grid of $m \times b$ blocks, and block (j, k) is the outer product $\mathbf{L}_{jk:} \mathbf{R}_{kj:}^\top$. A *Monarch* matrix swaps the two components of the row index:

$$\mathbf{M}_{(lj)(ki)} = \mathbf{L}_{jkl} \mathbf{R}_{kji}, \quad (lj) = (l-1)b + j. \quad (4)$$

Two properties matter:

- **Storage.** $\mathbf{L}$ and $\mathbf{R}$ hold $N(m+b)$ entries in total, which is $2N\sqrt{N}$ for $m = b = \sqrt{N}$.
- **Multiplication.** For $\mathbf{V} \in \mathbb{R}^{N \times d}$, $\mathbf{M}\mathbf{V}$ is two batches of small matmuls,

$$\mathbf{Y}_{jkv} = \sum_i \mathbf{R}_{kji} \mathbf{V}_{(ki)v}, \quad (\mathbf{M}\mathbf{V})_{(lj)v} = \sum_k \mathbf{L}_{jkl} \mathbf{Y}_{jkv}, \quad (5)$$

costing $\Theta(N(m+b)d) = \Theta(N\sqrt{N}d)$ when $m = b = \sqrt{N}$.

An exact projection of the attention matrix onto Monarch matrices would cost $O(N^2\sqrt{N})$ and require the full attention matrix. MonarchAttention avoids both by optimizing the variational objective directly over the factors.

Derivation

By Equation 2 applied row-wise (with $c = 1$), softmax attention is the solution of

$$\begin{aligned} \text{softmax}(\mathbf{Q}\mathbf{K}^\top) &= \arg \max_{\mathbf{P} \in \mathbb{R}^{N \times N}} \sum_{mn} \mathbf{P}_{mn} \mathbf{Q}_{mv} \mathbf{K}_{nv} - \sum_{mn} \mathbf{P}_{mn} \log \mathbf{P}_{mn} \\ \text{s.t.} \quad &\mathbf{P}_{mn} \geq 0, \quad \sum_n \mathbf{P}_{mn} = 1. \end{aligned} \quad (6)$$

We now constrain $\mathbf{P}$ to be Monarch, $\mathbf{P}_{(lj)(ki)} = \mathbf{L}_{jkl} \mathbf{R}_{kji}$ with $\mathbf{L} \in \mathbb{R}^{b \times m \times m}$ and $\mathbf{R} \in \mathbb{R}^{m \times b \times b}$, which gives

$$\begin{aligned} \max_{\mathbf{L}, \mathbf{R}} \quad &\sum_{jkl} \mathbf{L}_{jkl} \mathbf{R}_{kji} \mathbf{Q}_{(lj)v} \mathbf{K}_{(ki)v} - \sum_{jkl} \mathbf{L}_{jkl} \mathbf{R}_{kji} \log(\mathbf{L}_{jkl} \mathbf{R}_{kji}) \\ \text{s.t.} \quad &\mathbf{L}_{jkl} \mathbf{R}_{kji} \geq 0, \quad \sum_{k,i} \mathbf{L}_{jkl} \mathbf{R}_{kji} = 1. \end{aligned} \quad (7)$$

The goal is to rewrite this objective so that maximizing over $\mathbf{L}$ (or $\mathbf{R}$) alone looks exactly like Equation 2, giving closed-form alternating updates. This takes two steps: separate the constraints, then separate the entropy.

Separable constraints

We replace the constraints in Equation 7 with the equivalent *separable* form

$$\left\{ L_{jkl} \geq 0, \quad \sum_k L_{jkl} = 1 \right\} \quad \text{and} \quad \left\{ R_{kji} \geq 0, \quad \sum_i R_{kji} = 1 \right\}. \quad (8)$$

This is without loss of generality: any (L, R) satisfying the original constraints can be renormalized to satisfy Equation 8 without changing the products $L_{jkl} R_{kji}$, so the two constraint sets parametrize exactly the same Monarch matrices and the global optimum is preserved.

Separable entropy

Under Equation 8 the entropy term also separates. Splitting $\log(L_{jkl} R_{kji}) = \log L_{jkl} + \log R_{kji}$ (valid on non-negative entries with $0 \log 0 = 0$) and using $\sum_i R_{kji} = 1$ in the first resulting sum,

$$\begin{aligned} & - \sum L_{jkl} R_{kji} \log(L_{jkl} R_{kji}) \\ &= - \sum L_{jkl} \log L_{jkl} - \sum_{k,j} \left(\sum_l L_{jkl} \right) \left(\sum_i R_{kji} \log R_{kji} \right). \end{aligned} \quad (9)$$

This is the chain rule for the entropy of a factorized distribution: the entropy of the joint over (k, i) equals the entropy of the block choice plus the expected entropy of the within-block choice.

Update for L

Fix R and consider maximizing over one row $L_{j:l} \in \mathbb{R}^m$. Collecting the terms of Equation 7 and Equation 9 that depend on it,

$$\begin{aligned} \max_{L_{j:l}} \quad & \sum_{k,i,v} L_{jkl} R_{kji} Q_{(lj)v} K_{(ki)v} - \sum_k L_{jkl} \log L_{jkl} - \sum_k L_{jkl} \left(\sum_i R_{kji} \log R_{kji} \right) \\ \text{s.t.} \quad & L_{jkl} \geq 0, \quad \sum_k L_{jkl} = 1. \end{aligned} \quad (10)$$

Grouping the linear terms as a single score per k ,

$$\max_{L_{j:l}} \quad \sum_k L_{jkl} \underbrace{\left(\sum_v Q_{(lj)v} \sum_i R_{kji} K_{(ki)v} - \sum_i R_{kji} \log R_{kji} \right)}_{z_k} - \sum_k L_{jkl} \log L_{jkl}, \quad (11)$$

which is exactly Equation 2 with temperature $c = 1$. Hence

$$L_{jkl} = \text{softmax}_k \left(\sum_v Q_{(lj)v} \sum_i R_{kji} K_{(ki)v} - \sum_i R_{kji} \log R_{kji} \right), \quad (12)$$

where softmax_k is applied along k with all other indices fixed. The score for block k is the query's inner product with the R -weighted average key of that block, minus the negative entropy of the within-block distribution, so blocks with more spread-out attention receive a bonus.

Update for R

Fix L and consider one row $R_{k:j} \in \mathbb{R}^b$. The L -entropy term is constant, and the remaining terms are

$$\begin{aligned} \max_{R_{k:j}} \quad & \sum_{l,i,v} L_{jkl} R_{kji} Q_{(lj)v} K_{(ki)v} - \left(\sum_l L_{jkl} \right) \left(\sum_i R_{kji} \log R_{kji} \right) \\ \text{s.t.} \quad & R_{kji} \geq 0, \quad \sum_i R_{kji} = 1. \end{aligned} \quad (13)$$

Writing $c_{kj} = \sum_l L_{jkl}$ and grouping the linear terms,

$$\max_{\mathbf{R}_{kj}} \sum_i \mathbf{R}_{kji} \left(\sum_v \mathbf{K}_{(ki)v} \sum_l L_{jkl} \mathbf{Q}_{(lj)v} \right) - c_{kj} \sum_i \mathbf{R}_{kji} \log \mathbf{R}_{kji}, \quad (14)$$

which is Equation 2 with temperature c_{kj} . When $c_{kj} > 0$,

$$\mathbf{R}_{kji} = \text{softmax}_i \left(\frac{1}{c_{kj}} \sum_v \mathbf{K}_{(ki)v} \sum_l L_{jkl} \mathbf{Q}_{(lj)v} \right). \quad (15)$$

Here the score for key i is its inner product with the L -weighted sum of the queries in grid column j that route to block k , and the temperature is the total query mass c_{kj} routed to that block. When $c_{kj} = 0$ every $L_{jkl} = 0$, the objective does not depend on $\mathbf{R}_{kj}$, and it may be left unchanged.

Optimization

Write $f(\mathbf{R})$ for the L -update Equation 12 and $g(\mathbf{L})$ for the $\mathbf{R}$ -update Equation 15. Both are exact maximizers of Equation 7 over one factor with the other fixed. There are two natural ways to iterate them.

Gauss-Seidel (alternating). Each iteration uses the freshest factor,

$$\mathbf{L}^{t+1} = f(\mathbf{R}^t), \quad \mathbf{R}^{t+1} = g(\mathbf{L}^{t+1}). \quad (16)$$

This is block coordinate ascent with two blocks. Every update is an exact maximization over its block, so the objective is non-decreasing, and because the objective is bounded above on the constraint set the sequence of objective values converges. Per iteration, the two updates are sequential: the $\mathbf{R}$ -update cannot start until the L -update finishes.

Jacobi (simultaneous). Each iteration uses only the previous iterate,

$$\mathbf{L}^{t+1} = f(\mathbf{R}^t), \quad \mathbf{R}^{t+1} = g(\mathbf{L}^t). \quad (17)$$

The two updates are independent and can run in parallel. The price is that $(\mathbf{L}^{t+1}, \mathbf{R}^{t+1})$ is not the result of any single exact block maximization, so the objective need not increase monotonically.

Jacobi is two interleaved Gauss-Seidel chains

Substituting Equation 17 into itself,

$$\mathbf{R}^{t+2} = g(f(\mathbf{R}^t)), \quad \mathbf{L}^{t+2} = f(g(\mathbf{L}^t)). \quad (18)$$

The map $g \circ f$ is exactly one Gauss-Seidel iteration Equation 16 written in terms of $\mathbf{R}$, and $f \circ g$ is the same iteration written in terms of $\mathbf{L}$ with the update order swapped. Hence the even Jacobi iterates $\mathbf{R}^0, \mathbf{R}^2, \mathbf{R}^4, \dots$ are precisely the Gauss-Seidel iterates started from $\mathbf{R}^0$, and the odd Jacobi iterates $\mathbf{R}^1, \mathbf{R}^3, \dots$ are the Gauss-Seidel iterates started from $\mathbf{R}^1 = g(\mathbf{L}^0)$. The same holds for $\mathbf{L}$. Jacobi therefore runs two independent Gauss-Seidel chains, each advancing by one step every two Jacobi iterations.

Two consequences follow immediately.

- **Convergence.** Each chain inherits the monotone convergence of Gauss-Seidel. The Jacobi pair $(\mathbf{L}^t, \mathbf{R}^t)$ converges if and only if both chains reach the same fixed point; otherwise it settles into a period-two cycle between two stationary points, which Gauss-Seidel cannot do.
- **Rate.** If Gauss-Seidel contracts the objective gap by a factor ρ_{GS} per iteration near a fixed point, Jacobi contracts it by ρ_{GS} every two iterations, i.e. by

$$\rho_J = \sqrt{\rho_{\text{GS}}} \quad (19)$$

per iteration. This is the classical relation $\rho_{\text{GS}} = \rho_J^2$ for two-block (consistently ordered) systems, here obtained without linearization because the two-block structure makes Equation 18 exact.

Cost per iteration

Both schemes evaluate f and g once per iteration, so the arithmetic cost is identical: $\Theta(N(m+b)d)$ per iteration by the same accounting as Equation 5. What differs is the critical path, measured in serial evaluations of f or g . After s serial evaluations, Gauss-Seidel holds one factor that has just been updated and one that is a step stale: at $s = 2T$ its $\mathbf{L}$ was computed at depth $2T - 1$. Jacobi, if f and g are co-scheduled, completes s iterations in the same depth, and since $\mathbf{L}^s = f(\mathbf{R}^{s-1})$ and $\mathbf{R}^s = g(\mathbf{L}^{s-1})$, both of its factors are at full depth s . By Equation 19 the two schemes share the same asymptotic rate per unit depth, $\rho_J^2 = \rho_{GS}$, so this freshness advantage shows up as a better constant rather than a better rate, and it is largest when the depth budget is small. The extreme case is a budget of a single evaluation: Gauss-Seidel can only take a half step, $(f(\mathbf{R}^0), \mathbf{R}^0)$, which leaves $\mathbf{R}$ at its data-independent initialization, whereas one Jacobi step $(f(\mathbf{R}^0), g(\mathbf{L}^0))$ adapts both factors to $\mathbf{Q}$ and $\mathbf{K}$. The price is twice the total work for a given depth, since by Equation 18 Jacobi is running a second Gauss-Seidel chain, and the latency advantage only materializes if the device has spare capacity to run f and g concurrently; a large batched update that already saturates the device gains nothing. Gauss-Seidel remains preferable when total work is the constraint and guarantees monotone ascent without two-cycles; Jacobi is preferable when latency is the constraint and parallel capacity is available, most clearly for very few steps.

Numerical check

For $N = 256$, $m = b = 16$, $d = 32$ with Gaussian queries and keys scaled so that the scores have unit variance, and a shared random initialization, the objective gap to the converged value behaves as follows.

Iteration	GS gap	GS KL	Jacobi gap	Jacobi KL
1	3.4×10^0	111.71	6.5×10^0	114.79
2	7.9×10^{-2}	108.38	8.5×10^{-1}	109.15
4	5.0×10^{-4}	108.30	3.9×10^{-2}	108.34
6	9.3×10^{-6}	108.30	2.9×10^{-3}	108.31
10	1.3×10^{-8}	108.30	3.7×10^{-5}	108.30
20	$< 10^{-12}$	108.30	8.7×10^{-9}	108.30
30	$< 10^{-12}$	108.30	3.9×10^{-12}	108.30

Table 1: Objective gap and reverse KL to exact attention, summed over rows, for Gauss-Seidel (GS) and Jacobi iteration from the same initialization.

A least-squares fit of the geometric rate gives $\rho_{GS} \approx 0.165$ and $\rho_J \approx 0.409$, so $\rho_J^2 \approx 0.168 \approx \rho_{GS}$, matching Equation 19. Both schemes converge to the same fixed point here, and the Jacobi $\mathbf{R}$ -iterate at step $2t$ coincides with the Gauss-Seidel $\mathbf{R}$ -iterate at step t to machine precision, confirming Equation 18. Comparing at equal depth rather than equal iteration count (Gauss-Seidel iteration t against Jacobi iteration $2t$), Jacobi is ahead throughout, e.g. a gap of 8.5×10^{-1} versus 3.4×10^0 at depth two and 3.9×10^{-2} versus 7.9×10^{-2} at depth four. At depth one, over 20 random draws of $\mathbf{Q}$, $\mathbf{K}$, one Jacobi step beat the Gauss-Seidel half step in every draw, reducing the summed KL from 119.9 to 112.3 from a uniform initialization and from 213.5 to 129.1 from a random one. Note that a handful of Gauss-Seidel iterations already reach the KL floor of the Monarch class: the gap between the converged KL and the KL after two iterations is 10^{-2} , whereas the KL floor itself is 10^2 , so in practice the approximation error is dominated by the Monarch constraint, not by incomplete optimization.

Interpretation

The Monarch constraint models each attention row as a factorized distribution: from query (l, j) , first choose a key block k with probability $\mathbf{L}_{jkl}$, then a key i within that block with probability $\mathbf{R}_{kji}$. Because $\mathbf{R}_{kji}$ does not depend on l , all queries in grid column j share the same within-block key distributions but weight the blocks individually.

Using the identity $\sum_n \mathbf{p}_n z_n + H(\mathbf{p}) = \log \sum_n e^{z_n} - \text{KL}(\mathbf{p} \parallel \text{softmax}(\mathbf{z}))$ row by row and discarding the log-partition terms, which do not depend on $\mathbf{P}$, Equation 7 is equivalent to

$$\min_{\mathbf{L}, \mathbf{R}} \sum_{l,j} \text{KL}(\mathbf{P}_{(lj):} \parallel \mathbf{p}_{lj}^*), \quad \mathbf{p}_{lj}^* = \text{softmax}((\mathbf{Q}\mathbf{K}^\top)_{(lj):}), \quad (20)$$

so MonarchAttention finds the Monarch factorization closest to exact attention in reverse KL. Reverse KL is mode-seeking: it concentrates mass on the highest-attention keys.